

Documentation & Project Diary

Innovation Lab 2
Year 2026

Project: **2BUGG AI**

Team: **34**

1. General Information

Project name: 2BUGG_AI

Supervisor: FH-Prof. DI Dr. Lars Mehnen

Innovation Lab 2, summer term 2026

Project team:

Nikola Cvetkovic, if24b103@technikum-wien.at, Informatik, Project Manager

Krystian Kedzior, if24b272@technikum-wien.at, Informatik

Bercesta Yagmur Aslan Özugur, if24b157@technikum-wien.at, Informatik

1.1. Management Summary of the Project

This project analyzes whether AI tools can help detect and debug bugs in real software projects. In the previous semester we tested the idea on small example programs. In this semester we extend the approach to larger and more realistic projects from GitHub.

The main goal is to test how well an AI debugging tool works when applied to different programming languages and different types of bugs. We analyzed repositories, detected bugs, classified them by type, and observed how accurately the tool was able to locate and explain these problems.

The priorities of the project are:

1. Extend the existing debugging tool to support real-world repositories written in C, C++, Java, and Python, including multi-file projects with external dependencies and different build systems.
2. Test the tool on a set of GitHub repositories of different sizes and complexity levels.
3. Classify the detected bugs by type and difficulty.
4. Observe how the AI-generated analysis behaves on different types of repositories, use the results to drive further improvements to the tool, and document what worked and what did not.

The results of this project will show where AI-assisted debugging works well, where it struggles, and what would need to change to make it reliable enough for real-world use.

1.2. Framework Conditions and Project Environment

The project environment mainly consists of publicly available GitHub repositories. Around 50 to 60 repositories of different sizes were analyzed, including small, medium, and large projects. The repositories cover different programming languages, specifically C, C++, Java, and Python.

1.3. Semester-Roadmap

The work in this semester builds on the prototype developed in the previous semester. During the previous semester the project was carried out by a smaller team, which limited the scope of the experiments and the effort estimation. In this semester the team consists of three students, which allows a more structured analysis and a clearer distribution of tasks.

The following roadmap gives an overview of how we planned the semester at the beginning: (The actual sprint contents evolved during the semester based on feedback from the supervisor and the results of our test runs)

| 2buggAI Semester Roadmap |

Sprint 1

- **Topics:**
 - Review of the debugging tool prototype from the previous semester and analysis of the existing repository
 - Discussion and definition of the user stories for 4th semester
 - Initial search and selection of GitHub repositories
 - Analysis of those GitHub repositories' structure to understand how the debugging tool needs to be adapted for different repository types

Sprint 2

- **Topics:**
 - Collection and filtering of GitHub repositories of different sizes and programming languages
 - Cloning and organizing the selected repositories locally
 - Installation of required dependencies for the selected repositories
 - Identification of executable targets within the repositories
 - Extension of the debugging tool to support different repository structures, programming languages, and dependency setups

Sprint 3

- **Topics**
 - Further extension of the debugging tool to support multiple programming languages
 - Further development of support for repositories with multiple source files and dependencies
 - Improvement of the handling of large debugging outputs from complex projects

Sprint 4

- **Topics**
 - Integration of an additional AI model (Claude) for debugging analysis
 - Improvement of debug output parsing and filtering
 - Improvement of the CLI output to display debugging results more clearly
 - Categorization of detected bugs into groups
 - Analysis of dependency bugs, logic bugs, UI bugs, concurrency bugs, and Heisenbugs
 - Expansion of experiments to a larger number of repositories
 - Testing the debugging approach on more complex code bases

Sprint 5

- **Topics**
 - Further development and improvement of the debugging analysis workflow
 - Extended debugging experiments on additional repositories
 - Testing and validation of the debugging results

Sprint 6

- **Topics**
 - Analysis of the debugging results across different repository types and programming languages.
 - Finalization of project documentation

Sprint 7

- **Topics**
 - Finalization of project diary and remaining documentation
 - Preparation of final presentation slides
 - Recording of the final presentation video

Actual Sprint Outcomes vs. Planned Roadmap:

While the roadmap above reflects the initial planning, the actual sprint contents evolved based on testing results and feedback from the supervisor.

- **Sprint 1** was completed as planned. The prototype was reviewed, repositories were selected, and multi-file C/C++ compilation support was added.
- **Sprint 2** was largely completed as planned. The CLI output was improved and the codebase was restructured into proper classes. Many repositories had to be excluded because they used CMake, which the tool does not support.
- **Sprint 3** was completed as planned. The error checklist system, the AI fix loop, the CodeChanger, and Python and Java compilation support were all added.
- **Sprint 4** deviated from the plan. Claude AI integration was not implemented due to API access costs. Instead, the focus shifted to Java build system detection (Maven, Gradle, Spring Boot) and adding JDB and PDB debugger support.
- **Sprint 5** was completed as planned. Automatic checklist expansion, difficulty-based error sorting, and English prompt translation were added. A fix for interactive Python programs that caused pipeline deadlocks was also implemented.
- **Sprint 6** went beyond the original plan. In addition to finalizing documentation, full Python dependency management and automatic Java and Gradle version installation were implemented.
- **Sprint 7** was completed as planned. The project diary, presentation slides, and final video were all finalized.

1.4. Collaboration & Tooling:

Repository:

<https://github.com/bercesteyagmur/2bugqAI.git>

Development Tools:

- Programming Language: C++
- IDE: VS Code, CLion
- Version Control: Git, GitHub
- AI/LLM: OpenAI API

Team Communication:

- WhatsApp for daily communication

- Discord for Team discussions
- Meetings with supervisor
- Regular Sprint Reviews

1.5. Remarks

JavaScript/TypeScript support was considered during the planning phase but was excluded from the project scope, as the existing debugger infrastructure (GDB, Valgrind, JDB, PDB) does not cover JavaScript runtimes.

The integration of Claude as a second AI model was planned for Sprint 4 but was not implemented within the semester, as the OpenAI-based pipeline covered the required functionality.

The tool was developed and tested exclusively on Linux (Ubuntu) and WSL environments. Windows and macOS are not supported.

The initial plan was to analyze around 60 GitHub repositories. In practice, around 50 to 60 repositories were tested, as many repositories turned out to be unsuitable due to missing dependencies, unsupported build systems, or requiring external hardware or services that were not available in our environment.

Dependency version conflicts in Python projects, where packages listed in requirements.txt are incompatible with each other, cannot be resolved automatically by the tool. As discussed with the supervisor, solving this would require reimplementing significant functionality and is out of scope for this prototype.

CMake and npm are not supported as build systems. The tool handles gcc/g++ for C and C++, javac, Maven, and Gradle for Java, and pip with virtual environments for Python.

2. Brief Description of the Project

This project investigates whether AI-based debugging tools can help developers find bugs in real software projects. In the previous semester, Nikola and Krystian developed and tested a first prototype of an AI-assisted debugging system using small example programs and simple bug scenarios. The results were promising but the scope was limited. This semester we extended the prototype significantly and tested it on real open-source repositories from GitHub, covering projects of different sizes written in C, C++, Java, and Python.

2.1. Scope

The main goal of this semester was to extend the AI-assisted debugging tool from the previous semester and test it on real GitHub repositories. The tool takes a repository as input, collects all the source files, figures out the programming language and build system, compiles the project, runs the right debugger, classifies the errors it finds, and sends everything to the OpenAI API.

The AI then explains what the bug is and suggests a fix, which the tool can write directly into the source file.

We supported four programming languages this semester: C, C++, Java, and Python. We tested the tool on around 50 to 60 real repositories from GitHub, ranging from small projects to larger ones with multiple source files, external dependencies, and different build systems.

Some things were out of scope. We did not add JavaScript or TypeScript support because our debugger setup does not cover JavaScript runtimes. We also did not try to automatically resolve dependency version conflicts in Python, since that would have been a much bigger task than what we could fit into the semester. The tool only runs on Linux and WSL - not on Windows or macOS.

2.2. Greatest Challenges

The most significant challenge was the complexity of real-world software repositories. Unlike the small example programs we worked with in the previous semester, real open-source projects from GitHub often contain dozens of source files spread across multiple folders, external dependencies that need to be installed before the project can even be compiled, and different build systems that each require a completely different approach.

Handling all of this automatically turned out to be much harder than writing the core debugging logic itself. For Java projects, we had to detect whether a project used plain javac, Maven, or Gradle, and in some cases Spring Boot on top of that. For Gradle projects, we also had to figure out which Gradle version the project required and download it automatically, because using the wrong version would break the build. On top of that, many Java projects required a specific JDK version (for example Java 8 or Java 11) and if the wrong version was installed the compilation would simply fail. We had to detect this from the build files and install the correct JDK version automatically before even trying to compile. For Python projects, we had to create a virtual environment per project, detect which packages were needed even when no requirements.txt existed, and deal with cases where the package name used in import statements does not match the actual pip package name, for example, cv2 is installed as opencv-python.

On top of all that, many problems only showed up when we actually ran the tool on real

repositories. Something that looked like a simple project would fail in a way we did not expect at all. Sometimes a system library was missing, sometimes a build file was not in the root folder but buried somewhere inside a subfolder, sometimes a Python program just sat there waiting for user input and the whole pipeline got stuck. Every time we fixed one thing, the next repository had a different problem we had not seen before. It felt like the edge cases never stopped coming.

2.3. Added Value for Users

Setting up a debugging workflow manually is time-consuming. A developer normally has to collect the relevant source files, figure out how to compile the project, run the right debugger, read through potentially large and messy output, try to understand what went wrong, and then come up with a fix. For unfamiliar codebases or projects in a language the developer is less comfortable with, this can take a long time even before any actual fixing starts.

Our tool automates all of this. The developer just points it at a repository, provides a short description of the problem, and the tool takes care of everything else, such as collecting files, detecting the language and build system, installing dependencies, compiling, running the debugger, filtering the output, classifying the errors, and sending everything to the AI for analysis. The result is a structured explanation of what went wrong and a suggested fix, which the tool can also apply automatically.

This is especially useful when working with unfamiliar repositories or when the error output is large and hard to read. Instead of spending time just understanding what the debugger is telling you, the developer can focus on deciding whether the suggested fix makes sense.

2.4. Approach

The implementation was carried out across seven sprints. Each sprint extended the tool with new capabilities, which were then immediately tested on real GitHub repositories. The findings from each test run were used to drive further improvements in the following sprint.

2.5. Expected Deliverables

- An improved and extended version of the AI-assisted debugging tool supporting C, C++, Java, and Python
- A documented collection of debugging experiments performed on real GitHub repositories
- Project documentation including sprint reports, effort estimation, and a final presentation video

2.6. Evaluation / Experiments

Our debugging tool was tested on a set of real open-source GitHub repositories of different sizes and programming languages. For each repository, the tool collected runtime output, classified detected errors, and forwarded the results to the OpenAI API for analysis. The results were documented and evaluated.

Each test run was also used to identify weaknesses in the tool and to drive further improvements such as extending the package name mapping table, improving dependency detection, and refining the error classification system.

The detailed test results, including per-repository outputs, bug classifications, and AI analysis results, are documented in the project repository under the Documentations folder, in the file named “2buggAI_Test_Results_Analysis.pdf” available at:
<https://github.com/bercesteayagmur/2buggAI.git>

3. Specification of the Solution

3.1. System Environment (System Boundaries)

The solution developed in this project is a prototype AI-assisted debugging system that analyzes program code and debugging outputs. The system is implemented as a command-line application and serves as a debugging support tool for developers.

The prototype developed in the previous semester provides the basic functionality for collecting debugging information and analyzing it with a large language model. In this semester, the existing prototype will be extended and improved. The system will be applied to real software repositories in order to evaluate the effectiveness of AI-supported debugging in more complex software projects.

The system includes the following components:

- Recursive collection and processing of source code files from project directories
- Automatic detection of the programming language based on file extensions
- Automatic detection of the build system and execution of the appropriate build command (gcc/g++ for C/C++, javac/Maven/Gradle for Java, Python virtual environment for Python)
- Automatic installation of project dependencies (pip packages for Python, JDK version management for Java, Gradle version download)
- Execution of external debugging tools: GDB and Valgrind for C/C++, JDB for Java, PDB for Python
- Collecting and structuring debugging output from runtime execution
- Automatic classification of detected bugs by type and difficulty using an internal error checklist
- Automatic expansion of the error checklist based on newly discovered bug categories
- Creating structured debugging reports in JSON format
- Sending debugging reports to the OpenAI API for AI-based bug analysis
- Automatic application of AI-suggested fixes to source files, with a .bak backup created before any change
- Displaying structured debugging explanations to the user in the terminal

The system does not include the following components:

- Development environments (IDEs)
- Operating systems
- Automatic resolution of dependency version conflicts between packages
- Support for JavaScript or TypeScript projects
- Execution of real-world production applications

The system processes external artifacts such as source code files and debugging outputs but is not responsible for creating these artifacts.

3.2. Features (Functional Requirements)

The functional requirements are described as epics and user stories. Some core functionality was already implemented in the previous semester. In this semester, the prototype will be extended and applied to larger real-world software repositories.

Epic 1: Execution and Debugging Data Collection

User Story 1

As a developer, I want to provide a program or repository to the system so that it can be analyzed by the debugging tool.

User Story 2

As a developer, I want the system to detect how a program should be built and executed so that repositories with different build systems can be analyzed.

User Story 3

As a developer, I want the system to run debugging tools such as GDB so that stack traces and runtime errors can be collected.

User Story 4

As a developer, I want the system to run Valgrind so that memory errors and invalid memory accesses can be detected.

User Story 5

As a developer, I want debugging outputs to be filtered and parsed so that relevant information can be extracted from large debugging logs.

Epic 2: AI-Based Bug Explanation

User Story 6

As a developer, I want the debugging outputs collected from tools such as GDB, Valgrind, JDB, and PDB to be combined into a structured JSON report so that the information can be analyzed by an AI model.

User Story 7

As a developer, I want the system to send the generated debugging report to an AI language model so that the cause of the bug and possible fixes can be explained.

User Story 8

As a developer, I want the AI model to suggest possible fixes for the detected bug and automatically apply them to the source files, so that the correction process is faster and a backup of the original file is created before any change is made.

Epic 3: Running of Real Software Repositories

User Story 9

As a developer, I want the debugging tool to analyze real-world software repositories so that the system can be tested on realistic projects.

User Story 10

As a developer, I want the system to process repositories of different sizes (small, medium, and large) so that the debugging tool can work on various project scales.

User Story 11

As a developer, I want the system to analyze repositories written in different programming languages such as Python, Java, C and C++ so that the debugging tool can support multiple programming environments.

User Story 12

As a developer, I want the debugging tool to work with repositories that contain multiple source files and external dependencies so that larger real-world projects can be analyzed correctly.

User Story 13

As a developer, I want to run the debugging tool on approximately 60 GitHub repositories so that the system can be tested on a sufficiently large dataset.

Epic 4: Bug Classification and Evaluation

User Story 14

As a developer, I want the system to automatically categorize detected bugs into different bug types such as dependency bugs, logic bugs, memory errors, and name errors, so that the most critical issues can be prioritized and fixed first based on their difficulty level.

User Story 15

As a developer, I want to evaluate the debugging results from each tested repository so that the strengths and limitations of the AI-based analysis can be documented and understood.

User Story 16

As a developer, I want to observe how the tool behaves on repositories of different sizes and complexity levels so that the practical limits of the debugging pipeline can be identified and documented.

3.3. Interfaces

Our system communicates with the following external interfaces:

OpenAI API: Our tool sends structured JSON debugging reports to the OpenAI REST API over HTTPS. The request includes the collected source code, error output, and bug description. The response contains the AI-generated bug explanation and suggested fix. To keep things secure, the API key is stored as an environment variable and never hardcoded anywhere in the project.

File System: Our tool reads source code files from the local file system and writes results back to it. Before any fix is applied, a .bak backup of the original file is created automatically. Optionally, the full debugging report can be saved as a JSON file using --json-out.

External Debuggers: Our tool runs the following external debuggers as separate processes and collects their output:

- GDB -> for runtime debugging of C and C++ programs
- Valgrind -> for memory error detection in C and C++ programs
- JDB -> for runtime debugging of Java programs
- PDB -> for runtime debugging of Python programs

3.4. Quality Characteristics and Technical Requirements

Usability:

The tool is operated entirely via the command line. No graphical interface is required. The user provides a path, a short description, and optional flags. The terminal output is structured and readable, showing file collection results, compilation status, debugger output, error

classification, and AI analysis in a clear sequence.

Performance:

The analysis time depends on the size of the repository, the number of dependencies, and the response time of the OpenAI API. For small projects the full pipeline typically completes within one to two minutes. For larger repositories with many dependencies the installation and build phase take significantly longer.

Portability:

The tool runs on Linux (Ubuntu) and WSL. It is not designed for Windows or macOS.

Reliability:

If a step in the pipeline fails, for example compilation or dependency installation, the tool continues with the remaining steps rather than stopping completely. This ensures that at least partial results and AI-based static analysis are always produced even when the project cannot be fully executed.

Security:

The OpenAI API key is read from an environment variable and is never written to disk.

4. Effort Estimation

After the first meeting with our supervisor, in which the goals and expectations for this semester were discussed, we started to estimate the expected effort for the project in an additional team meeting. In addition, we studied the self-study materials provided on Moodle as well as the estimation methods that were explained during the in-person InnoLab class.

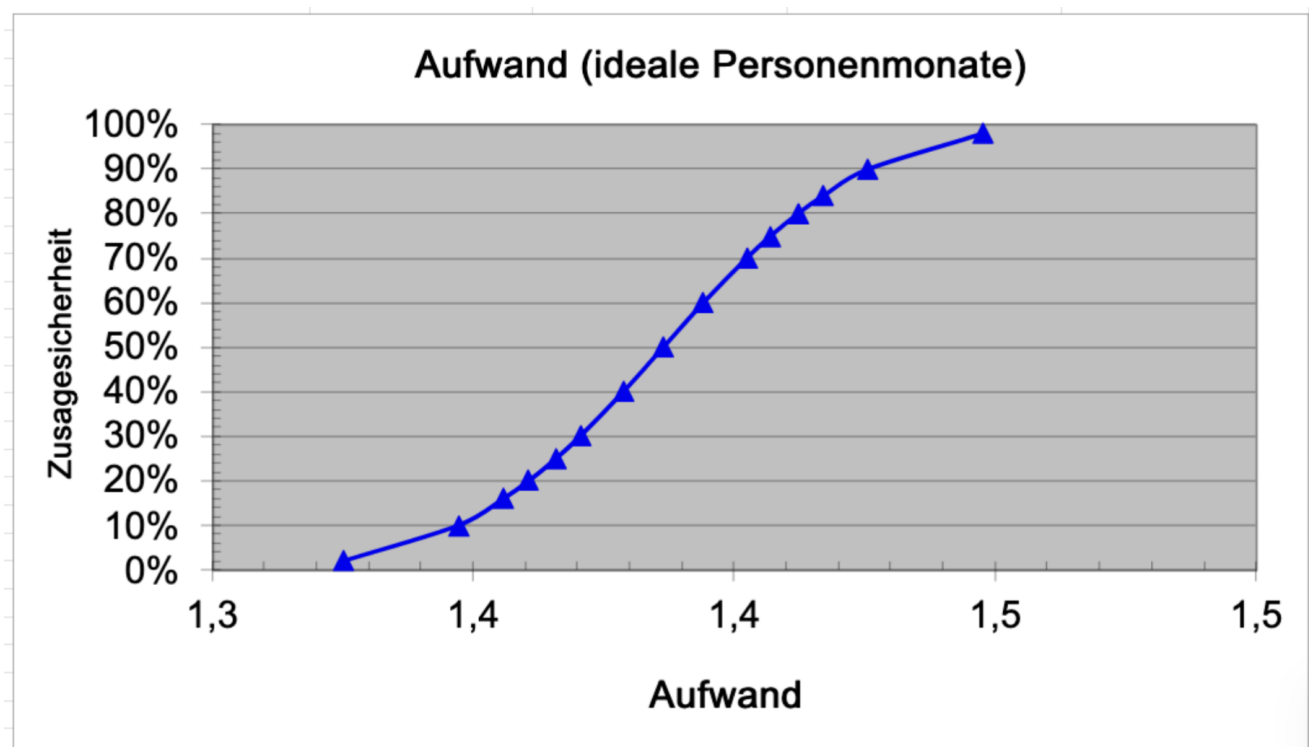
During this internal meeting, we identified the main work packages that are necessary for the project. These work packages were based on the planned extension of the AI debugging prototype developed in the previous semester and the new goal of analyzing real GitHub repositories with different programming languages and bug types.

After defining the work packages, we discussed the expected effort for each task. The estimation was based on the expected complexity of the work packages. Tasks such as running debugging experiments on many repositories, working with different programming languages, installing dependencies, and processing large debugging outputs were considered when estimating the required effort. Based on this estimation, the work packages will be distributed across the project sprints for this semester.

The results provided by the estimation tool were afterwards reviewed and discussed within the team (see the screenshots below). One important observation is that the expected effort at the 50% confidence level is approximately 221.83 hours. Our team plans to spend around 225 hours in total on the project during the semester. (Our project team consists of three students, $75 \times 3 = 225$ hours). This means that the planned effort is very close to the calculated expected value. At a higher confidence level of 98%, the estimation increases to 231.65 hours.

Delphi Verfahren mit PERT - Unsicherheit ausdrückbar durch Schätzbereich: Breite, p(range)													
ID	Themes / Areas / Arbeitspakete Toplevel	Epics / Arbeitspakete TopLevel	User Stories / Detail Level / Beschreibung	Optimistisch (Sp)	Wahrscheinlich (Sp)	Pessimistisch (Sp)	Dokumentation von Annahmen, Überlegungen, Risiken, Bedingungen und Diskussionsergebnissen aus dem Schätzmeeting	Erwartet nach PERT 1:4:1 (Sp)	Kalibrierter erwarteter Aufwand (Ph)	p(range) (5-100)	Divisor (0,25-6)	Standard-Abweichung (Ph)	Varianz
				162,0	219,0	293,0		221,83	221,83			4,907	24,083
1	Administratives	Project diary preparation	Prepare the first sections of the project diary for the first submission and maintain it during the semester	10	14	18		14,00	14,00	99,73	6,00	1,333	1,778
2	Administratives	Sprint planning	Create an initial sprint plan for the semester and refine the plan before each sprint	6	10	14	Sprint plan may change depending on implementation progress.	10,00	10,00	99,73	6,00	1,333	1,778
3	Administratives	Effort estimation preparation (DAGoPERT)	Prepare the DAGoPERT estimation table and organize work packages	5	8	12	Estimation based on team discussion and previous semester experience.	8,17	8,17	99,73	6,00	1,167	1,361
4	Research Setup	Prototype Review	New team member reviews and understands the existing debugging prototype	3	5	8	Only required for the newly joined team member	5,17	5,17	99,73	6,00	0,833	0,694
5	Dataset Prep.	Repository search and selection	Search GitHub and select approximately 100 repositories based on language diversity, size.	12	15	20		15,33	15,33	99,73	6,00	1,333	1,778
6	Dataset Prep.	Repository dataset preparation	Organize the selected GitHub repositories and store repository information in a CSV file	4	6	8		6,00	6,00	99,73	6,00	0,667	0,444
7	Dataset Prep.	Repository cloning	Clone the selected repositories locally	4	6	8	Some repositories may fail to clone due to missing access or large size.	6,00	6,00	99,73	6,00	0,667	0,444
8	Dataset Prep.	Dependency installation	Install required dependencies for the selected repositories	4	6	8		6,00	6,00	99,73	6,00	0,667	0,444
9	Dataset Prep.	Executable target detection	Determine which file or executable inside each repository should be analyzed by the debugging tool	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
10	System Extension	Multi-programming-language extension	Extend the tool so programs written in Python, Java, C++ and JavaScript can be analyzed	14	18	22		18,00	18,00	99,73	6,00	1,333	1,778
11	System Extension	Claude AI integration	Integrate the Claude AI API in addition to the existing GPT-based analysis.	8	12	14	Requires access to the Claude API	11,67	11,67	99,73	6,00	1,000	1,000
12	System Extension	Large debug output handling	Improve the system to process debugging outputs from big projects	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
13	System Extension	Multi-file repository debugging	Extend the system to work with larger repositories containing multiple source files and dependencies.	23	28	33		28,00	28,00	99,73	6,00	1,667	2,778
14	System Extension	Repository build system detection	Detect the build system used in each repository (e.g. npm, pip, Maven, CMake) so that the debugging tool can run the correct build and execution	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
15	Output Processing	Debug output filtering	Filter and extract the most relevant information from large debugging outputs	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
16	Output Processing	Debug output parsing improvement	Improve the parsing of debugging outputs from tools such as GDB and Valgrind so that complex outputs from larger projects can be handled more	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
17	CLI / User Interface	CLI result visualization	Improve the CLI output so debugging results, detected bugs are displayed in a clearer and more structured way.	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
18	Experiment Execution	Repository debugging workflow testing	Execute integration tests on selected repositories with different programming languages, project sizes, and dependency setups to verify that the debugging tool can compile, run, collect debugging outputs, and generate AI results.	13	17	20		16,83	16,83	99,73	6,00	1,167	1,361
19	Bug Classification	Bug categorization	Define bug categories and assign detected bugs to the defined bug categories and record the result	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
20	Documentation		Document the dataset, debugging workflow and results	6	8	10		8,00	8,00	99,73	6,00	0,667	0,444
21	Documentation	Final report and presentation	Prepare the final project report, presentation and video	8	10	14		10,33	10,33	99,73	6,00	1,000	1,000

Aufwandschätzung mit Zusagesicherheit (Konfidenzintervall)				
Zusagesicherheit (% Konfidenz)	Aufwand (Personenstunden)	Aufwand (Personentage brutto)	Aufwand (Personenmonate brutto)	Aufwand (Personenjahre brutto)
2%	212,02	26,50	1,33	0,11
10%	215,55	26,94	1,35	0,11
16%	216,93	27,12	1,36	0,11
20%	217,71	27,21	1,36	0,11
25%	218,55	27,32	1,37	0,11
30%	219,28	27,41	1,37	0,11
40%	220,61	27,58	1,38	0,11
50%	221,83	27,73	1,39	0,12
60%	223,06	27,88	1,39	0,12
70%	224,39	28,05	1,40	0,12
75%	225,12	28,14	1,41	0,12
80%	225,96	28,24	1,41	0,12
84%	226,74	28,34	1,42	0,12
90%	228,11	28,51	1,43	0,12
98%	231,65	28,96	1,45	0,12



Cost Estimation Based on Estimated Effort:

Angebotskalkulation				
Parameter zur Angebotskalkulation				
Stundensatz / Tagsatz				
Stundensatz (EUR) ²⁾	139,00			
Stunden pro Tag ¹⁾	7,70			
Tagsatzsatz (EUR) ²⁾	1070,30			
1) Umrechnungsfaktor "Stunden pro Tag" gilt für die Kostenberechnung, nicht für die Ressourcenberechnung!				
2) Der Tagsatz bzw. Stundensatz ist bei dieser einfachen Kalkulation ein Mischsatz für das Unternehmen bzw. das Projekt (inkl. Gemeinkosten, etc.)!				
Vereinfachte Angebotskalkulation				
	25% Konfidenz	50% Konfidenz	75% Konfidenz	98% Konfidenz
Gesamtaufwand (h)	218,55	221,83	225,12	231,65
Gesamtaufwand (Tage)	28,38	28,81	29,24	30,08
Zusatzkosten 1 (EUR)	€ 0,00	€ 0,00	€ 0,00	€ 0,00
Zusatzkosten 2 (EUR)	€ 0,00	€ 0,00	€ 0,00	€ 0,00
Gesamtkosten (EUR)	€ 30.377,80	€ 30.834,83	€ 31.291,87	€ 32.199,11
Bemerkung zur Zusagesicherheit (% Konfidenz)	Nicht für Angebot geeignet - nur für den internen Gebrauch!	Bestenfalls bei strategischem Projekt als Angebot verwendbar.	Bei gut bekannten Projekttypen für Angebot einsetzbar.	Gut als Basis für Angebot einsetzbar.

Based on the estimated effort, a simple cost calculation was made using the parameters from the tool. The calculation uses an hourly rate of €139 and 7.7 working hours per day, which results in a daily rate of about €1070.30. Depending on the confidence level, the estimated total project cost is between about €30,377 (25% confidence) and €32,199 (98% confidence). This estimation can also be used to calculate a potential project price if the work were done for a paying customer.

4.1. Adapted Effort Estimation

We revised our effort estimation mid-semester based on what we actually experienced during implementation.

One of the bigger changes was Claude AI integration. We originally planned to integrate the Anthropic Claude API alongside GPT, but we ended up not implementing it because of the API access costs. We did research it and discuss it as a team, but it did not make sense to pay for API access just for this project.

We also increased the estimates for multi-programming-language support and multi-file repository debugging, because these turned out to be significantly more work than we initially thought. For example, supporting Python required handling virtual environments and pip installations, Java needed us to deal with both Maven and Gradle with different version requirements, and making the tool work with larger multi-file repositories meant we had to rethink how we collect and pass source files to the compiler and debugger. These were things we underestimated at the beginning of the semester.

Delphi Verfahren mit PERT - Unsicherheit ausdrückbar durch Schätzbereich: Breite, p(range)													
ID	Themes / Areas / Arbeitspakete Toplevel	Epics / Arbeitspakete TopLevel	User Stories / Detail Level / Beschreibung	Optimistisch (Sp)	Wahrscheinlich (Sp)	Pessimistisch (Sp)	Dokumentation von Annahmen, Überlegungen, Risiken, Bedingungen und Diskussionsergebnissen aus dem Schätzmeeting	Erwartet nach PERT 1:4:1 (Sp)	Kalibrierter erwarteter Aufwand (Ph)	p(range) (5-100)	Divisor (0,25-6)	Standard-Abweichung (Ph)	Varianz
				166,0	230,0	301,0		231,17	231,17			5,199	27,028
1	Administratives	Project diary preparation	Prepare the first sections of the project diary for the first submission and maintain it during the semester	10	16	18		15,33	15,33	99,73	6,00	1,333	1,778
2	Administratives	Sprint planning	Create an initial sprint plan for the semester and refine the plan before each sprint	6	10	14	Sprint plan may change depending on implementation progress.	10,00	10,00	99,73	6,00	1,333	1,778
3	Administratives	Effort estimation preparation (DAGoPERT)	Prepare the DAGoPERT estimation table and oranzize work packages	5	8	12	Estimation based on team discussion and previous semester experience.	8,17	8,17	99,73	6,00	1,167	1,361
4	Research Setup	Prototype Review	New team member reviews and understands the existing debugging protolove	3	5	8	Only required for the newly joined team member	5,17	5,17	99,73	6,00	0,833	0,694
5	Dataset Prep.	Repository search and selection	Search GitHub and select approximately 100 repositories based on language diversity, size.	12	15	20		15,33	15,33	99,73	6,00	1,333	1,778
6	Dataset Prep.	Repository dataset preparation	Organize the selected GitHub repositories and store repository information in a CSV file	4	6	8		6,00	6,00	99,73	6,00	0,667	0,444
7	Dataset Prep.	Repository cloning	Clone the selected repositories locally	4	6	8	Some repositories may fail to clone due to missing access or large size.	6,00	6,00	99,73	6,00	0,667	0,444
8	Dataset Prep.	Dependency installation	Install required dependencies for the selected repositories	4	6	8		6,00	6,00	99,73	6,00	0,667	0,444
9	Dataset Prep.	Executable target detection	Determine which file or executable inside each repository should be analyzed by the debuagino tool	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
10	System Extension	Multi-programming-language extension	Extend the tool so programs written in Python, Java, C++ and C can be	18	25	28		24,33	24,33	99,73	6,00	1,667	2,778
11	System Extension	Claude AI integration	Integrate the Claude AI API in addition to the existing GPT-based analysis.	2	3	5	Requires access to the Claude API	3,17	3,17	99,73	6,00	0,500	0,250
12	System Extension	Large debug output handling	Improve the system to process debugging outputs from big projects	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
13	System Extension	Multi-file repository debugging	Extend the system to work with larger repositories containing multiple source files and dependencies.	25	33	38		32,50	32,50	99,73	6,00	2,167	4,694
14	System Extension	Repository build system detection	Detect the build system used in each repository (e.g. npm, pip, Maven, CMake) so that the debugging tool can run the correct build and execution	7	9	13		9,33	9,33	99,73	6,00	1,000	1,000
15	Output Processing	Debug output filtering	Filter and extract the most relevant information from laroe debuagino outputs	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
16	Output Processing	Debug output parsing improvement	Improve the parsing of debugging outputs from tools such as GDB and Valgrind so that complex outputs from laroser coniects can be handled more	6	8	12		8,33	8,33	99,73	6,00	1,000	1,000
17	CLI / User Interface	CLI result visualization	Improve the CLI output so debugging results, detected bugs are displayed in a clearer and more structured way.	7	9	13		9,33	9,33	99,73	6,00	1,000	1,000
18	Experiment Execution	Repository debugging workflow testing	Execute integration tests on selected repositories with different programming languages, project sizes, and dependency setups to verify that the debugging tool can compile, run, collect debugging outputs, and generate AI	13	17	20		16,83	16,83	99,73	6,00	1,167	1,361
19	Bug Classification	Bug categorization	Define bug categories and assign detected bugs to the defined bug categories and report the result	8	12	16		12,00	12,00	99,73	6,00	1,333	1,778
20	Documentation		Document the dataset, debugging workflow and results	6	8	10		8,00	8,00	99,73	6,00	0,667	0,444
21	Documentation	Final report and presentation	Prepare the final project report, presentation and video	8	10	14		10,33	10,33	99,73	6,00	1,000	1,000

4.2. Interpretation of Results

The adapted DAGoPERT estimation gave us a total expected effort of around 231.17 hours at the 50% confidence level and 241.85 hours at the 98% confidence level. Our actual effort across all seven sprints was approximately 228 hours, which is slightly under the 50% estimate. This means we ended up being a bit more efficient than we expected, and we stayed within our planned budget of 225 hours.

When we look at individual sprints, most tasks went roughly as planned. The biggest time overruns happened in Sprint 2 and Sprint 6, mainly because testing on real GitHub repositories turned out to be a lot more unpredictable than we thought. Dependencies were missing, build systems behaved differently than expected, and some repositories simply did not work. These are things that are hard to estimate in advance.

Overall, we managed to complete all planned tasks across all seven sprints without any failed items. Even when some sprints took longer than expected, we were able to catch up and finish everything within the available time. We think this shows that our initial estimation was reasonable, and the DAGoPERT method helped us plan the semester in a realistic way.

5. Delivery

5.1. Final Solution and Source Code

The delivered solution is the 2BUGG_AI debugging tool, implemented as a command-line application in C++.

The full source code is available in the GitHub repository at:

<https://github.com/bercesteayagmur/2buggAI.git>

The repository contains the following components:

- main.cpp - main entry point and analysis pipeline
- src/ - all source files for each component
- includes/ - all header files
- errorchecklist.txt - the error category checklist used for bug detection
- makefile - build configuration
- Guide.md - installation and usage guide
- docs/ - project documentation, sprint reports, and test results

5.2. System Architecture and Data Storage

The system follows a three-layer architecture:

Input Layer (CLI):

- Accepts a path to a file or repository directory, a fix description, and optional flags such as --gdb, --valgrind, -r (recursive), -v (verbose), and --json-out.

Processing Layer:

- FileCollector -> recursively collects source files from the repository
- LanguageDetector -> detects the programming language from file extensions
- Compiler -> compiles C/C++ (gcc/g++), Java (javac/Maven/Gradle), or sets up Python virtual environments
- EnvironmentManager / DependencyManager -> manages Python virtual environments and dependency installation
- JavaProjectDetector -> detects Java project type (Plain, Maven, Gradle, Spring Boot)
- GdbRunner / JdbRunner / PdbRunner / ValgrindRunner -> runs the appropriate debugger
- ChecklistReader / ErrorMatcher / ErrorCollector -> detects and classifies errors
- OpenAIClient -> sends reports and fix requests to the OpenAI API
- CodeChanger -> writes AI-suggested fixes back to source files
- ReportJson -> assembles a structured JSON debugging report

Output Layer:

- Results are displayed in the terminal.
- Optionally a full JSON report can be saved to disk using --json-out. Before any fix is applied, a .bak backup of the original file is created automatically.

The system does not use a database. All data is stored as local files: source code files, the error

checklist (errorchecklist.txt), backup files (.bak), and optional JSON reports.

5.3. Required Licenses and Third-Party Components

- libcurl – MIT/curl License -> used for HTTP communication with the OpenAI API
- nlohmann/json -> MIT License, used for JSON parsing and generation
- GDB (GNU Debugger) -> .GPL v3, used for runtime debugging of C and C++ programs
- Valgrind -> GPL v2, used for memory error detection in C and C++ programs
- JDB (Java Debugger) -> GPL v2 with Classpath Exception (part of the JDK), used for runtime debugging of Java programs
- PDB (Python Debugger) -> PSF License (part of the Python standard library), used for runtime debugging of Python programs
- OpenAI API -> Commercial license, requires an API key, used for AI-based bug analysis and fix suggestions.

5.4. Hardware Specifications

The tool has no special hardware requirements. It was developed and tested on standard student laptops. The development environment used was CLion with the WSL (Windows Subsystem for Linux) extension, which allowed the team to write, build, and run the C++ tool directly inside a Linux environment from Windows without needing a separate Linux machine.

5.5. Installation

Step 1 - Install System Dependencies

Open a terminal and run:

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y \  
  build-essential \  
  gdb \  
  valgrind \  
  libcurl4-openssl-dev \  
  nlohmann-json3-dev \  
  git \  
  python3 \  
  default-jdk \  
  maven \  

```

The following are installed automatically by the tool at runtime and do not need to be installed manually:

- python3-pip -> installed automatically if missing before any Python project is analyzed
- python3-venv -> installed automatically when creating a virtual environment for a Python project
- Gradle -> the required Gradle version is downloaded automatically from services.gradle.org using wget and extracted with unzip. This is why wget and unzip must be present beforehand.
- Specific Java JDK versions (8, 11, 17, 21) -> installed automatically with apt when a Java project requires a version different from the default

Step 2 - Set Up the OpenAI API Key

1. Create an account at <https://platform.openai.com>
2. Generate an API key under API Keys
3. Add the key to your shell environment permanently

Step 3 - Download and Build the Tool

Clone the repository:

git clone <https://github.com/bercesteyagmur/2buggAI.git>

Then:

cd 2buggAI_innoLab

Build the tool:

Make

Run the Tool:

- In WSL, Windows paths are accessed via /mnt/.
- Basic syntax: ./buggy /mnt/<path> <fix-description> [options]

6. Our Project Diary

Project Kick-off Meeting with Supervisor (03.03.2026)

Participants:

Nikola Cvetkovic

Krystian Kedzior

Bercesta Yagmur Aslan Özugur

Our supervisor: FH-Prof. DI Dr. Lars Mehnen

Topics covered:

- Discussion of the debugging tool prototype developed in the 3rd semester
- Clarification of the project scope and expectations for the 4th semester
- Discussion about extending the debugging tool to analyze larger real-world repositories in Github
- Discussion about supporting repositories written in different programming languages such as Python, JavaScript/TypeScript, Java, and C++

Effort Estimation Meeting (05.03.2026)

Participants:

Nikola Cvetkovic

Krystian Kedzior

Bercesta Yagmur Aslan Özugur

Topics covered:

- Discussion and definition of the main user stories for the 4th semester
- Identification of the work packages based on the defined user stories
- Each team member estimated the required effort for every work package individually and provided optimistic, most likely, and pessimistic values.
- The individual estimations were compared and discussed within the team.
- Based on the discussion, final optimistic, most likely, and pessimistic estimates for all work packages are defined.
- Entering these values into the DAGσPERT estimation tool to calculate the total estimated effort.

Sprint 1 - Looking Back and Starting Fresh

Period: 10.03.2026- 24.03. 2026

What we did:

- Went through the prototype built by Nikola and Krystian in the previous semester together as a team and figured out what already worked and what was missing.
- Bercesta joined the team as a new member and spent time understanding the existing codebase.
- Searched for and selected GitHub repositories to work with. We aimed for a good mix of small, medium, and large projects in Python, Java, and C++

- Analyzed which changes the tool would need in order to support real C and C++ repositories. The old prototype could only handle a single source file, but real projects consist of many files across multiple folders that all need to be compiled together. We worked through what this meant in practice: the tool first needed to search through the entire repository and collect all relevant source files, while being careful to skip folders that contain generated build output rather than actual code. It also needed to detect where the header files were located and pass that information to the compiler, since without the correct include paths the compilation would fail. Finally, the compiler call itself had to be extended to handle multiple files at once instead of just one. After working through these steps, the tool was able to take a full C or C++ repository, compile all its parts correctly.

Challenges:

- We already had a file collector that scanned the entire repository and gathered all the source files including the header files. But even though the tool could see all the files, the compilation was still failing because the compiler could not find the headers. We did not understand why at first because the files were right there. The problem was that collecting the files and telling the compiler where they are two completely different things. Just because our tool found the .h files did not mean the compiler knew where to look for them. We had to add -I flags to the compile command so the compiler also knew the exact directories. After that it worked.

What went well:

- The multi-file support for C and C++ came together well.

Sprint 1 Review Meeting with Supervisor (26.03.2026)

Participants:

Nikola Cvetkovic

Krystian Kedzior

Bercesta Yagmur Aslan Özugur

Our supervisor: FH-Prof. DI Dr. Lars Mehnen

Topics covered:

- Review of the initial search and selection of GitHub repositories across different sizes and programming languages.
- Agreement that the tool should support Python, Java (including Maven and Gradle projects), besides C/C++ projects this semester.

Sprint 2 - Extending and Optimizing the Foundation

Period: 24.03. 2026 – 14.04.2026

What we did

- Improved the CLI output for folder analysis: the tool now shows a clear summary at the start with the total number of files found and the target path, and then lists each file one by one with its line count and size. In verbose mode it also prints the full source code of each file in a formatted box so the user can see exactly what is

- being analyzed
- Ran the tests with multi-file compilation and recursive file detection on actual GitHub repositories
- Fixed issues in the code based on what the test outputs revealed
- Restructured the codebase - in the beginning almost all the logic was written directly inside main.cpp, which made it hard to read and extend. We moved everything into proper classes and separated the responsibilities.

Challenges:

- Finding suitable repositories for testing was harder than expected. Many repositories we found on GitHub required external libraries like OpenCV or Boost that were not easy to install, or they used CMake build systems that the tool did not support yet.

Sprint 2 Review Meeting with Supervisor (22.04.2026)

Participants:

Nikola Cvetkovic

Krystian Kedzior

Bercesta Yagmur Aslan Özugur

Our supervisor: FH-Prof. DI Dr. Lars Mehnen

- Our supervisor confirmed that multi-file compilation for C/C++ is complete and does not need further extension - the focus should now shift to adding Java and Python support
- We agreed to implement a checklist-based debugging approach where the system checks for common errors such as syntax errors, missing headers, memory leaks, and segmentation faults in a structured way
- It was discussed that after applying a fix the system should re-run the checks, because fixing one error can reveal new hidden errors

Sprint 3 - Checklists, Detection and Automatic Fixes

Period: 14.04.2026 - 28.04.2026

What we did:

- Implemented the error checklist system: a text file defines common and language-specific error types with keywords for each language. For example segmentation fault for C/C++ or NullPointerException for Java, SyntaxError for Python
- Implemented the ChecklistReader class to parse this file and the LanguageDetector class to automatically detect the programming language of a project from its file extensions
- Implemented the ErrorMatcher which compares the actual error output of a program against the checklist and identifies which known errors occurred
- Extended the OpenAIClient with a fix_code function that sends the detected error together with the source code to the AI and receives a corrected version back
- Extended main with a fix loop that goes through every detected error one by one, calls the fix function, and applies the result
- Implemented CodeChanger which writes the AI-suggested fix directly into the source file on disk - before overwriting, it creates a .bak backup of the original file so

nothing is lost if something goes wrong.

- Added Python and Java compilation support so the tool can now handle projects in these languages as well

Challenges:

- Adding Python and Java support was not something that could be done in one go. Every test run on a real repository revealed a new problem we had not thought of before, for example a different folder structure, a missing dependency, a build system we have not handled yet. It was a continuous cycle of testing, finding a new issue, fixing it, and testing again

What went well:

- Seeing the complete fix loop work end to end for the first time was a big moment. Our tool detected an error, sent it to the AI, and got a fix back and wrote the corrected code directly into the file automatically

Sprint 3 Review Meeting with Supervisor (13.05.2026)

Participants:

Nikola Cvetkovic

Krystian Kedzior

Bereste Yagmur Aslan Özugur (on sick leave)

Our supervisor: FH-Prof. DI Dr. Lars Mehnen

Topics covered:

- Presented the completed checklist system, the AI fix loop, and the automatic code writing functionality to the supervisor
- It was agreed that the next sprint should focus heavily on testing the system on real repositories
- The fix loop needed further improvement - specifically how new errors get added to the queue after a fix is applied
- A logging system for code changes was planned so the tool can track what was changed and when.
- A new -s flag for AI suggestions was discussed, giving the user the option to receive fix suggestions without the tool automatically applying them (similar to how -r works for recursive search).

Sprint 4 - Debugger Support and Testing

Period: 28.04.2026 - 12.05.2026

What we did:

- Previously the tool could only compile plain Java projects using a simple javac call. However, most real-world Java repositories on GitHub use Maven or Gradle

as their build system, which means a plain javac call simply does not work for them. In Sprint 4 we added JavaProjectDetector which recognizes the build system automatically - Maven, Gradle, Spring Boot Maven, Spring Boot Gradle, or plain Java - and calls the correct build command for each and also handles cases where these build files are not at the root but nested inside a subfolder

- Added JDB (Java Debugger) support for Java projects and PDB (Python Debugger) support for Python projects. Our tool now picks the right debugger automatically depending on the detected language, while C and C++ projects continue to use GDB as before
- Tested the tool on a variety of real repositories in Java, Python, and C/C++ and documented the outputs of each test run.
- Started researching how to optimize the error checklist based on what the test results revealed

Challenges and solution:

- One issue we ran into with the error checklist was that in the beginning every new error type had to be added manually to the text file. Every time we tested a new repository and saw an error the tool did not recognize, someone had to open the file and write a new entry by hand. This did not scale well as the number of tested repositories grew. We solved this by implementing a function that automatically appends new error categories discovered by the AI directly into the checklist file.

Sprint 4 Review Meeting (13.05.2026)

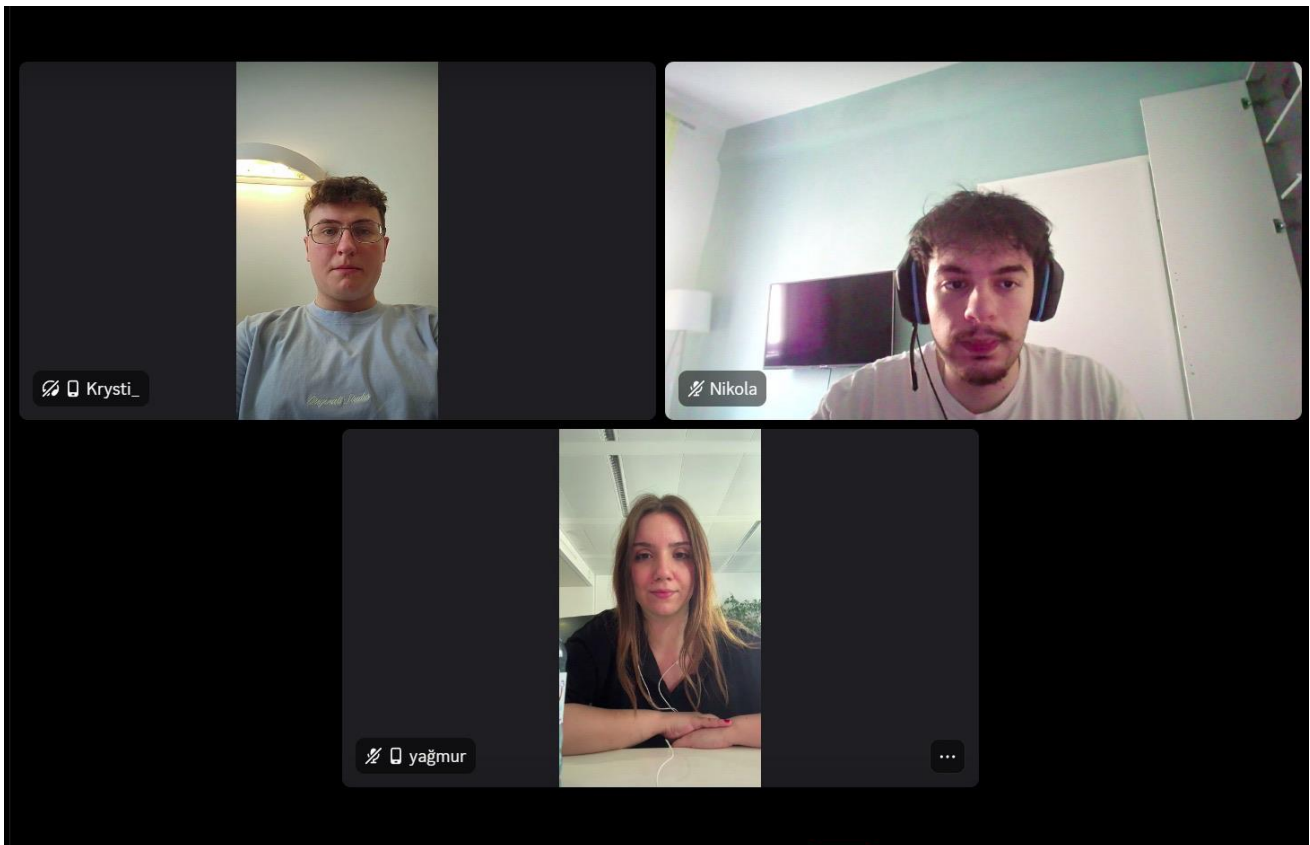
Participants:

- Nikola Cvetkovic
- Krystian Kedzior
- Berceste Yagmur Aslan Özugur

The scheduled review meeting with the supervisor was cancelled this sprint, so the team held an internal review meeting instead.

Topics covered:

- Reviewed the completed Java build system detection and confirmed that Maven, Gradle, Spring Boot Maven, Spring Boot Gradle, and plain Java projects are all handled correctly
- Planned the focus for Sprint 5: further testing on more repositories, improving error sorting by difficulty, and translating AI prompts to English to support all four languages correctly



Sprint 5 - Improvements and Extended Testing

Period: 12.05.2026 - 26.05.2026

What we did:

- Implemented the automatic checklist expansion: new error categories discovered by the AI during analysis are now added directly into the checklist file so the system learns from each run and does not need manual updates anymore
- Restructured the checklist format – the language is now its own separate column (language | name | type | keywords), which made the file cleaner and the parsing more reliable
- Translated all AI prompts from German to English so the tool works correctly for all four supported languages (C, C++, Java, Python).
- Improved the error collection logic: instead of fixing errors in whatever order they were found, errors are now sorted by difficulty first. Easy errors like syntax mistakes go to the front of the queue, medium errors like type mismatches come next, and harder errors like segmentation faults or memory corruption are handled last. This way the fix loop starts with the problems that are most likely to actually be fixable, rather than getting stuck on something complex right away.
- Performed additional testing on real repositories across all four languages and documented the results

Challenges:

- During testing we also discovered that some Python projects are interactive programs. They expect the user to type input during runtime, for example answering questions or choosing menu options. Our debugger cannot simulate user input, so when it ran these projects the program simply waited for input that never came, creating an infinite loop that blocked the entire pipeline. We identified this as a limitation and later implemented a fix that scans the source code for `input()` calls

before execution. If the code contains interactive input calls, the tool skips the execution step and proceeds directly to static AI analysis instead.

Sprint 5 Review Meeting with Supervisor (03.06.2026)

Participants:

- Nikola Cvetkovic
- Krystian Kedzior
- Berceste Yagmur Aslan Özugur
- Our supervisor: FH-Prof. DI Dr. Lars Mehnen

- Presented the test results and the improvements made during Sprint 5, including the automatic checklist expansion, the difficulty-based error sorting, and the extended Python and Java support
- We discussed the dependency version conflict problem encountered during Python testing - cases where packages in requirements.txt are incompatible with each other and cannot be resolved automatically
- Our supervisor confirmed that skipping these kinds of problems is the right approach for this project - solving dependency version conflicts automatically would require reimplementing significant functionality, searching for suitable libraries, and refactoring large parts of the project, which is far out of scope
- It was agreed that the final sprint should focus on finishing the remaining tests, completing the documentation, and preparing the final presentation.

Sprint 6 - Dependency Management and Environment Handling

Period: 26.05.2026 - 09.06.2026

What we did:

- For Python projects, implemented and improved:
 - Automatic pip detection and installation
 - Automatic virtual environment creation per project instead of using the global Python installation
 - Automatic dependency installation from requirements.txt
 - Dependency detection for projects without requirements.txt by scanning Python import statements directly from source files
 - A package name mapping for cases where the import name differs from the actual pip package name (for example cv2 -> opencv-python)
 - An internal ignore list for built-in Python modules so the tool does not try to install standard library modules like string or unittest with pip
 - Automatic removal of invalid or broken package names from requirements.txt when installation fails
- For Java projects, implemented and improved:
 - Automatic installation of the required Gradle version instead of relying on whatever is globally installed
 - Automatic detection and installation of the correct Java version when a project requires a specific one to compile correctly

What went well:

- The package name mapping and the built-in module ignore list solved a large number of false pip install attempts that had been causing noise in earlier test runs
- The automatic Gradle and Java version installation made Java testing significantly more reliable across different repositories

Team retrospective:

- We met one last time as a team to look back at the semester together. What surprised us most was how complex real open-source repositories actually are; handling dependencies, build systems, and version conflicts automatically turned out to be a much bigger challenge than writing the core debugging logic itself. If we did it again, we would start testing on real repositories earlier, because most of the hard problems only appeared during actual testing, not during development.

Sprint 6 Review Meeting with Supervisor (19.06.2026)

Participants:

Nikola Cvetkovic

Krystian Kedzior

Bereste Yagmur Aslan Özugur (on sick leave)

Our supervisor: FH-Prof. DI Dr. Lars Mehnert

Topics covered:

- Presented the final state of the debugging tool, including all improvements made during Sprint 6 such as automatic pip detection, virtual environment management, package name mapping, and the automatic Gradle and Java version installation.
- Our supervisor discussed a possible direction for the next semester: extending the system with a RAG (Retrieval-Augmented Generation) architecture. The idea is to introduce a database that stores the results of previous debugging sessions, such as detected errors, AI explanations, and suggested fixes. When a new project is analyzed, the system would first query this database to check whether a similar error was seen before and what was recommended. The AI would then use both the current project output and the retrieved past results to generate better and more informed bug explanations.

Sprint 7 - Documentation and Final Presentation

Period: 09.06.2026 - 23.06.2026

What we did:

- Finalized the project diary and all remaining project documentation
- Ran additional test runs on real repositories and used the outputs to identify remaining issues in the tool. Based on the results, we made further improvements to the code before finalizing the project.
- Prepared the final presentation slides
- Recorded the final presentation video